Cyberster Internship Program

Internship Report

WEEK 2: DETECTION, CUSTOM RULES, AND LOG ANALYSIS

Track: Blue Team
Name: Haris Zamir
Roll Number: CSI-B1-487
Instructor: Abdullah Zia

Table of Contents

Overview

During Week 2 of the Cyberster internship, the SOC environment underwent a significant shift — transitioning from a passive log-collection infrastructure to an active, real-time detection and automated response platform. The primary objectives this week were to configure File Integrity Monitoring (FIM) across both Linux and Windows endpoints, engineer custom XML detection rules aligned with MITRE ATT&CK techniques, conduct in-depth log analysis across the environment, and ultimately deploy automated Active Response to dynamically block attackers at the OS firewall level.

This report documents each task in detail, including the rationale, configuration steps, attack simulations, verification results, and key analytical takeaways.

Days 8 & 9 — File Integrity Monitoring (FIM)

File Integrity Monitoring (FIM) is a critical control for detecting unauthorized changes to system files, configuration files, and key directories. By monitoring the cryptographic hash values of files in real time, a SOC analyst can immediately identify whether a threat actor has tampered with important system components — a telltale sign of persistence or malware installation.

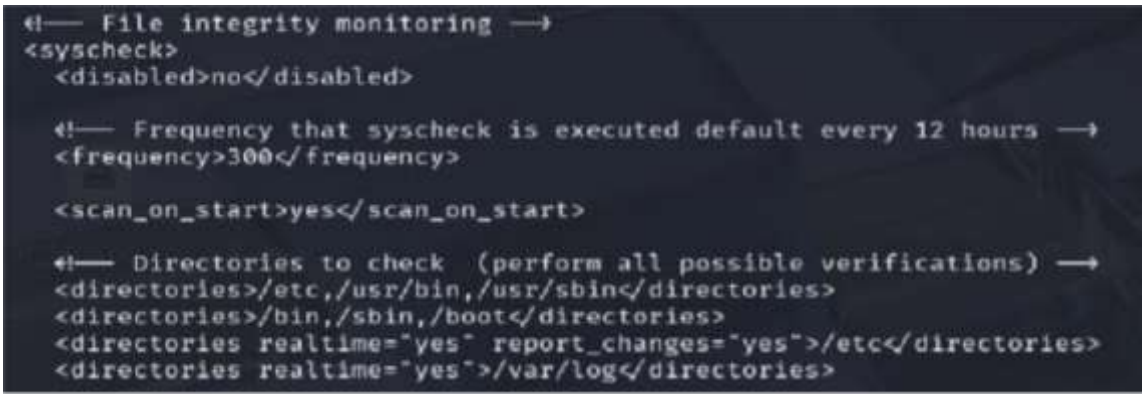
Linux FIM Configuration & Testing (Day 8)

To monitor the critical /etc directory on the Kali Linux agent — a prime target for attackers seeking to establish persistence — the local agent configuration file was modified to enable real-time reporting.

Step-by-Step Execution

- Opened the agent configuration file: sudo nano /var/ossec/etc/ossec.conf
- Added the /etc directory to the <syscheck> block with real-time reporting enabled: `<directories realtime="yes" report_changes="yes">/etc</directories>`

- Set the syscheck scan frequency to 300 seconds to ensure frequent baseline checks.
- Restarted the Wazuh agent to apply the changes: sudo systemctl restart wazuh-agent

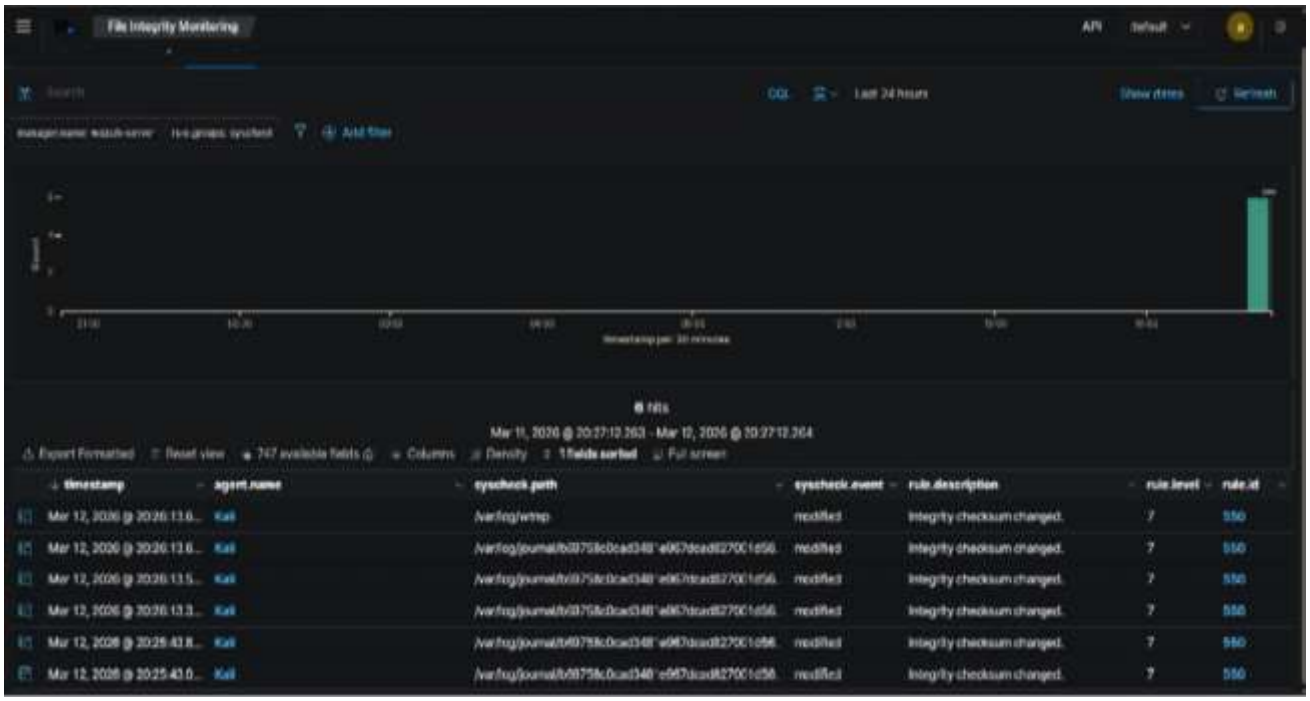


Attack & Verification

To validate the configuration was working correctly, an attacker establishing persistence was simulated by creating, modifying, and deleting a file inside the /etc directory.

- Command 1: sudo touch /etc/hacker_test.txt
- Command 2: echo "Attacker persistence established" | sudo tee /etc/hacker_test.txt
- Command 3: sudo rm /etc/hacker_test.txt

The Wazuh Dashboard immediately flagged all three operations, generating Level 7 "Integrity checksum changed" alerts for the Kali Linux agent. This confirmed the FIM configuration was operating as intended and catching file system changes in real time.



Windows 10 FIM Configuration & Testing (Day 9)

The same FIM process was replicated for the Windows 10 endpoint. This time, the target directory was C:\Windows\System32 — the location most commonly exploited by malware to drop malicious payloads, as it blends in with legitimate system files.

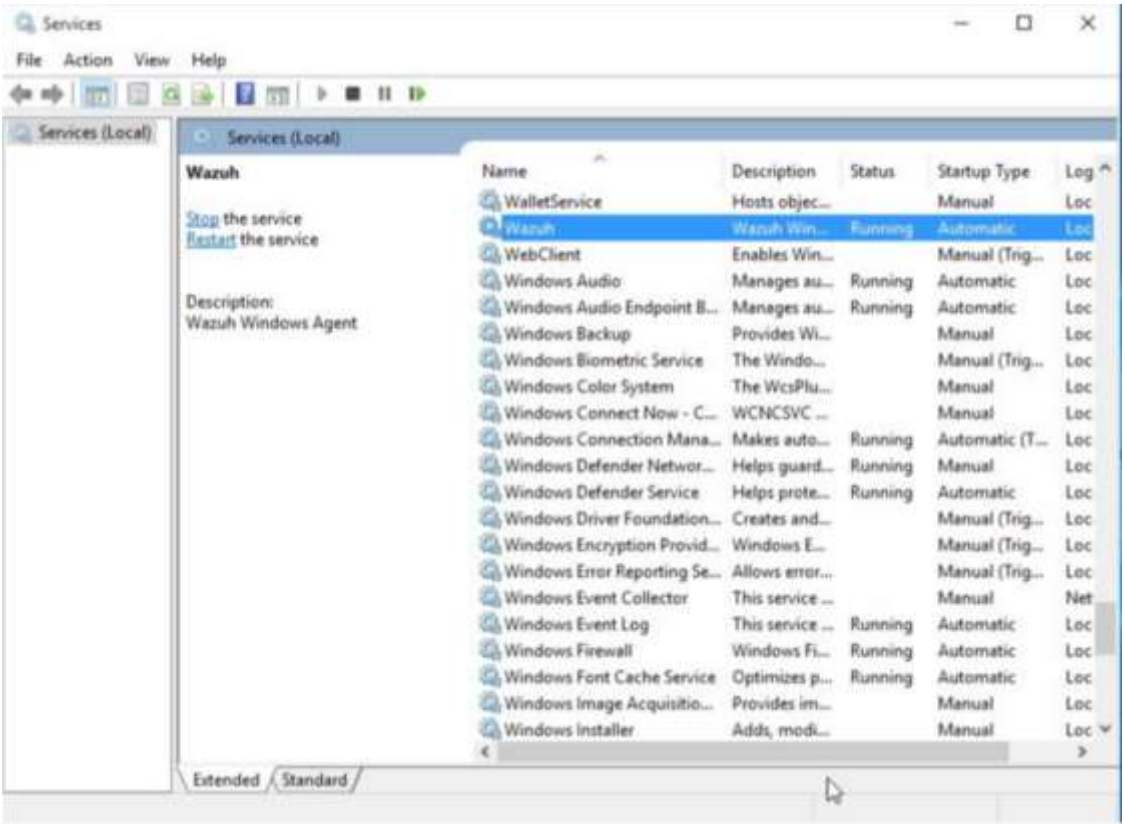
Step-by-Step Execution

- Opened C:\Program Files (x86)\ossec-agent\ossec.conf using Notepad running as Administrator.
- Configured the <syscheck> block to monitor the System32 directory:

```
<directories realtime="yes"
report_changes="yes">C:\Windows\System32</directories>
```

- Restarted the Wazuh Windows Agent via the Services Manager (services.msc).

```
<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>300</frequency>

<!-- Default files to be monitored. -->
<directories recursion_level="0" restrict="regedit.exe$|system.ini$|win.ini$">%WINDIR%</directories>

<directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|
<directories recursion_level="0">%WINDIR%\SysNative\drivers\etc</directories>
<directories recursion_level="0" restrict="WMIC.exe$">%WINDIR%\SysNative\wbem</directories>
<directories recursion_level="0" restrict="powershell.exe$">%WINDIR%\SysNative\WindowsPowerShell\v1.
<directories recursion_level="0" restrict="winrm.vbs$">%WINDIR%\SysNative</directories>

<!-- 32-bit programs. -->
<directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|
<directories recursion_level="0">%WINDIR%\System32\drivers\etc</directories>
<directories recursion_level="0" restrict="WMIC.exe$">%WINDIR%\System32\wbem</directories>
<directories recursion_level="0" restrict="powershell.exe$">%WINDIR%\System32\WindowsPowerShell\v1.0
<directories recursion_level="0" restrict="winrm.vbs$">%WINDIR%\System32</directories>

<directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup</directories
<directories realtime="yes" report_changes="yes">C:\Windows\System32</directories>

<ignore>%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup\desktop.ini</ignore>
```

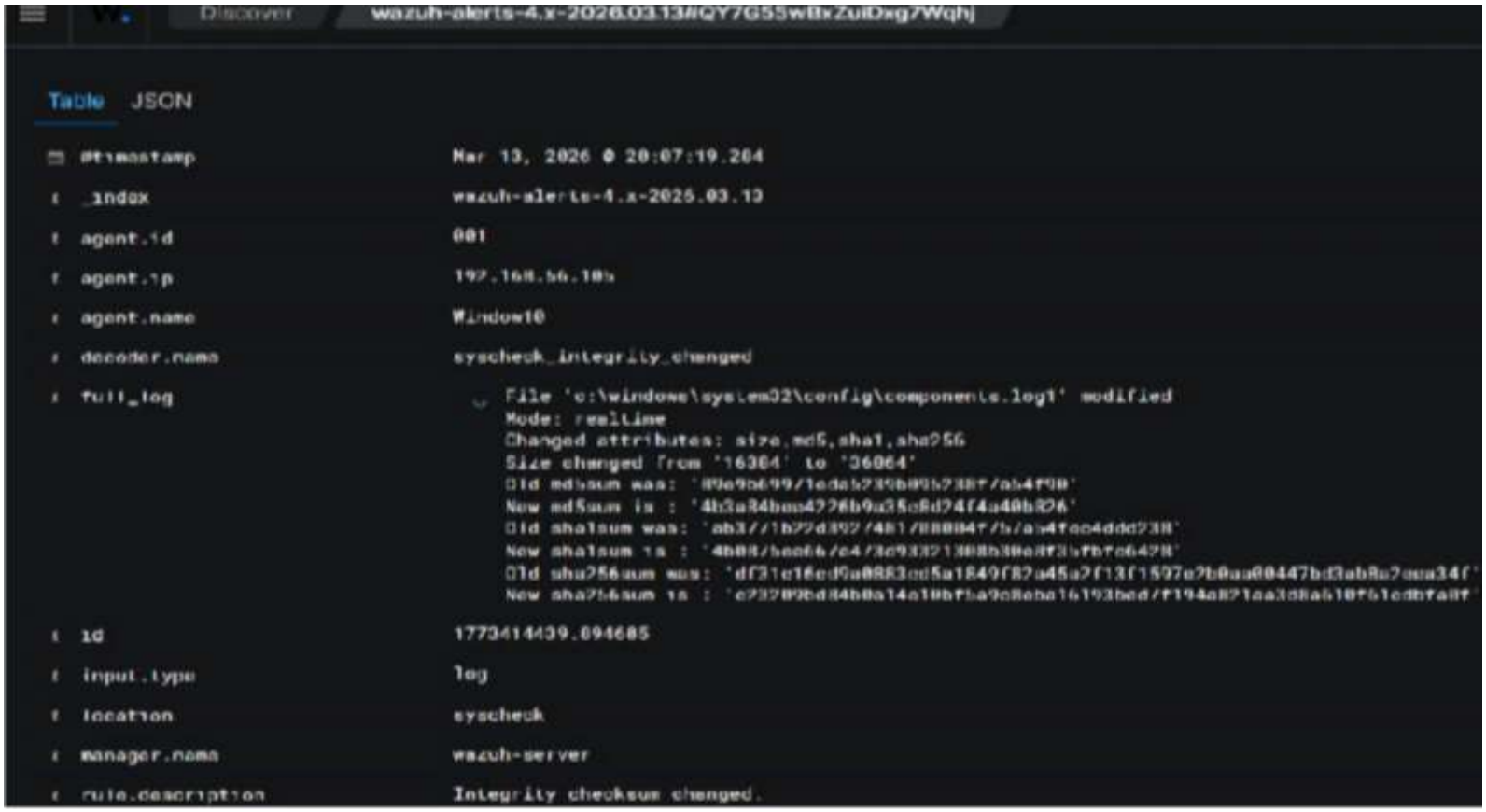


Attack & Verification

Using an elevated Windows Command Prompt, a simulated malware payload drop was executed against the monitored directory.

- Command 1: echo "Malicious payload dropped" > C:\Windows\System32\hacker_test.txt
- Command 2: echo "Updating payload" >> C:\Windows\System32\hacker_test.txt
- Command 3: del C:\Windows\System32\hacker_test.txt

Wazuh successfully captured all three file system events. Expanding the JSON alert data revealed the exact SHA256, MD5, and SHA1 hash changes of the modified files — providing forensic-quality evidence of the tamper activity.



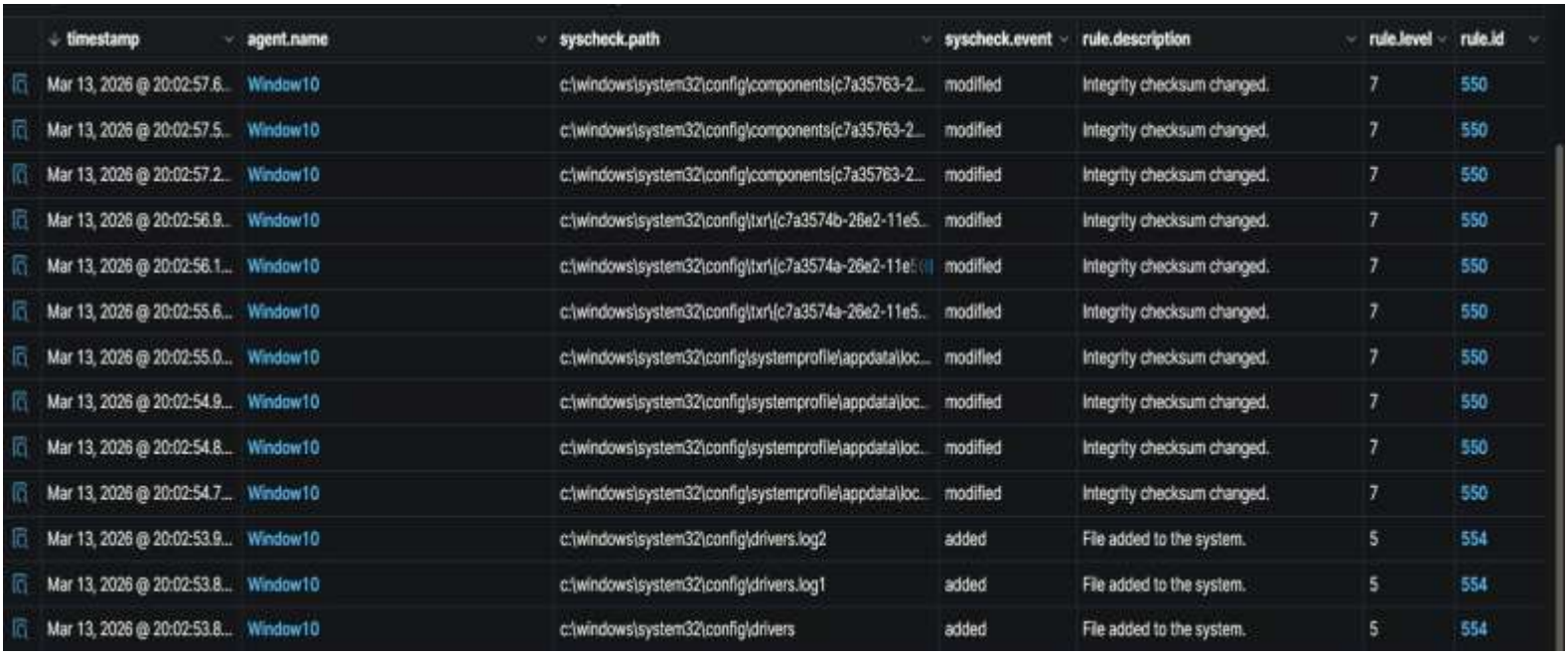

```
C:\Windows\system32>echo "Malicious payload dropped" > C:\Windows\System32\hacker_test.txt

C:\Windows\system32>echo "Updating payload" >> C:\Windows\System32\hacker_test.txt

C:\Windows\system32>del C:\Windows\System32\hacker_test.txt
```

Days 10 & 11 — Custom Detection Rules

Out-of-the-box SIEM rules are often too broad or produce excessive noise. This phase required engineering precise, custom XML detection rules designed to catch specific attacker behaviors mapped directly to the MITRE ATT&CK framework. Each rule targets a distinct technique and was validated through a live attack simulation.

Rule 1: New User Creation

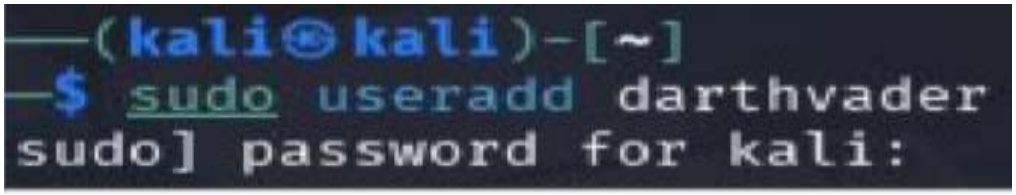
To detect unauthorized persistence via account creation (MITRE T1136 — Create Account), a custom rule (ID 100010) was engineered to fire a Level 10 Critical alert whenever a new user account is created on a Linux system.



timestamp	agent.name	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Mar 13, 2026 @ 20:02:57.6...	Window10	c:\windows\system32\config\components{c7a35763-2...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:57.5...	Window10	c:\windows\system32\config\components{c7a35763-2...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:57.2...	Window10	c:\windows\system32\config\components{c7a35763-2...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:56.9...	Window10	c:\windows\system32\config\txr\{c7a3574b-26e2-11e5...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:56.1...	Window10	c:\windows\system32\config\txr\{c7a3574a-26e2-11e5...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:55.6...	Window10	c:\windows\system32\config\txr\{c7a3574a-26e2-11e5...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:55.0...	Window10	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:54.9...	Window10	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:54.8...	Window10	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:54.7...	Window10	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity checksum changed.	7	550
Mar 13, 2026 @ 20:02:53.9...	Window10	c:\windows\system32\config\drivers.log2	added	File added to the system.	5	554
Mar 13, 2026 @ 20:02:53.8...	Window10	c:\windows\system32\config\drivers.log1	added	File added to the system.	5	554
Mar 13, 2026 @ 20:02:53.8...	Window10	c:\windows\system32\config\drivers	added	File added to the system.	5	554


```
<!-- Custom Rule 1: New User Creation -->
<rule id="100010" level="10">
 <if_sid>5902</if_sid>
 <description>CRITICAL: New user account created on the system</description>
 <mitre>
  <id>T1136</id>
 </mitre>
</rule>
```

Verification: The command sudo useradd darthvader was executed on the Kali Linux machine. The custom rule fired precisely as expected, generating a Level 10 Critical alert and confirming the rule's operational readiness.

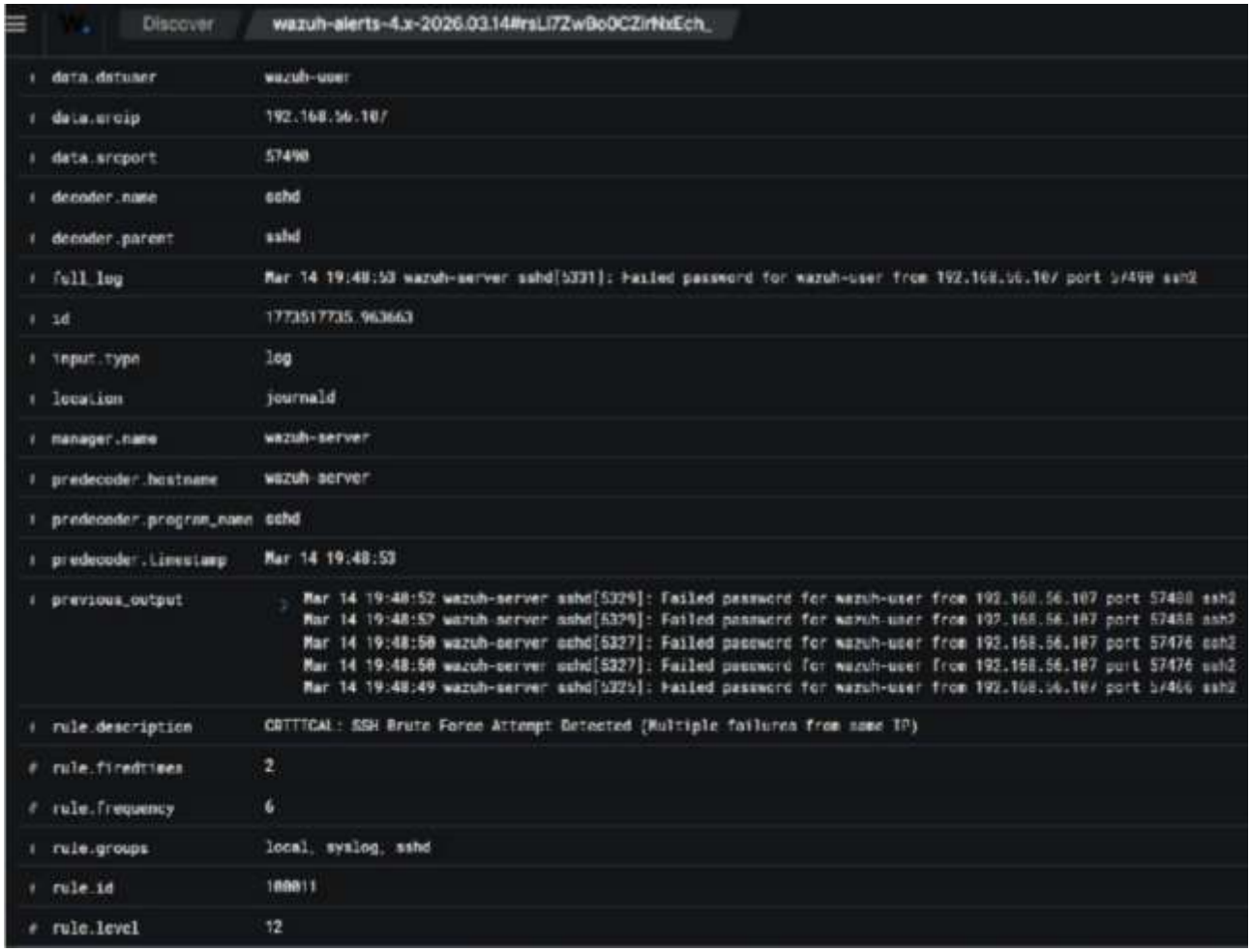


Rule 2: SSH Brute Force Detection

To detect credential stuffing and password-spraying attacks (MITRE T1110.001 — Brute Force: Password Guessing), a frequency-based rule (ID 100011) was created. This rule fires a Level 12 Critical alert when 6 or more SSH authentication failures occur from the same source IP address within a 120-second window, using the <if_matched_sid>5760</if_matched_sid> correlation mechanism.

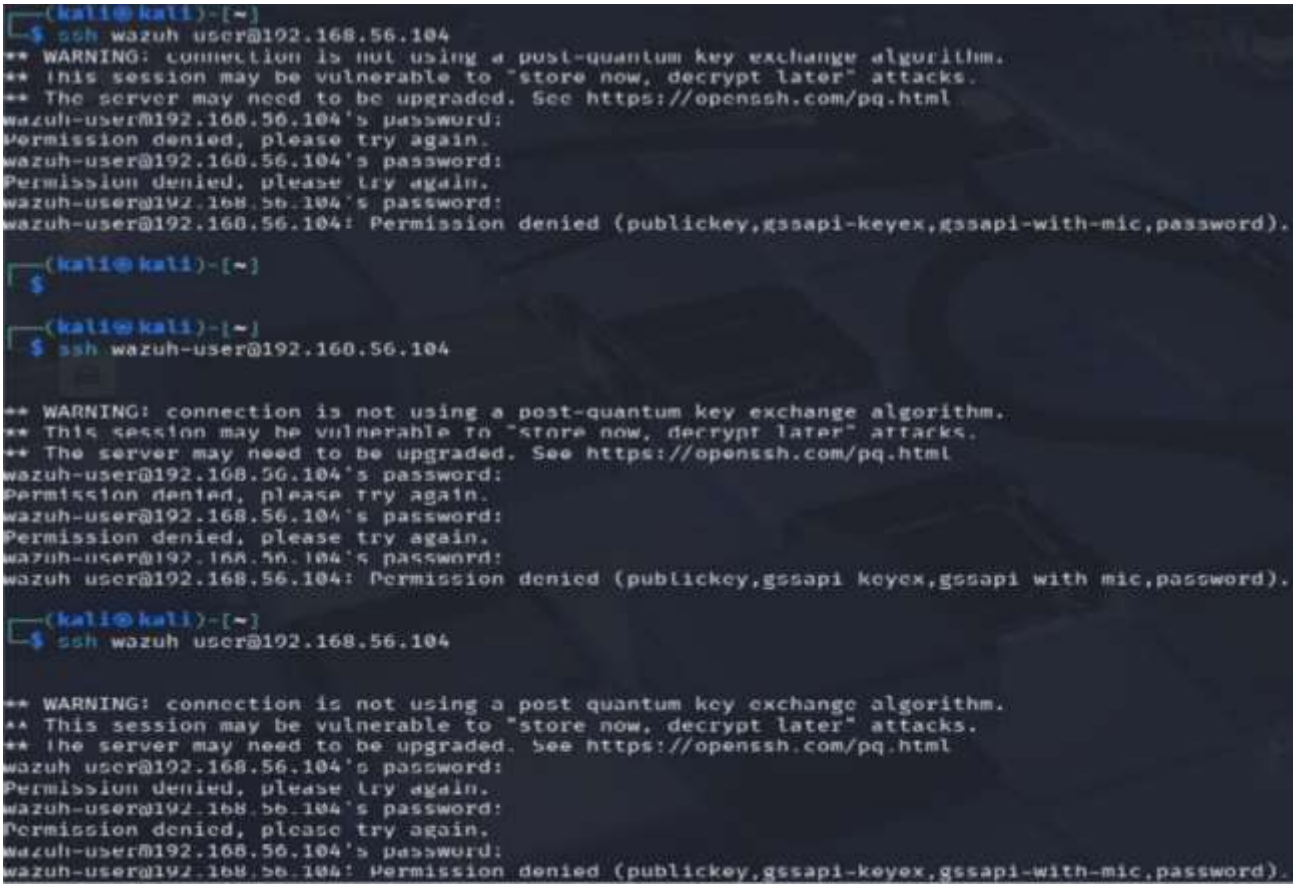
Verification: The SSH login prompt was intentionally spammed with multiple incorrect passwords from the Kali machine. Wazuh correctly grouped the correlated events and triggered the Level 12 Critical alert, confirming the rule was functioning correctly.

```
<!-- Custom Rule 2: SSH Brute Force -->
<rule id="100011" level="12" frequency="6" timeframe="120">
 <if_matched_sid>5760</if_matched_sid>
 <same_source_ip />
 <description>CRITICAL: SSH Brute Force Attempt Detected (Multiple failures from same IP)</description>
 <mitre>
  <id>T1110.001</id>
 </mitre>
</rule>
```


Discover
wazuh-alerts-4.x-2026.03.14#rsLi7ZwBo0CZirNxEch_
t data.dstuser wazuh-user
t data.srcip 192.168.56.107
t data.srcport 57490
t decoder.name sshd
t decoder.parent sshd
t full_log Mar 14 19:48:53 wazuh-server sshd[5331]: Failed password for wazuh-user from 192.168.56.107 port 57490 ssh2
t id 1773517735.963663
t input.type log
t location journald
t manager.name wazuh-server
t predecoder.hostname wazuh-server
t predecoder.program_name sshd
t predecoder.timestamp Mar 14 19:48:53
t previous_output Mar 14 19:48:52 wazuh-server sshd[5329]: Failed password for wazuh-user from 192.168.56.107 port 57488 ssh2
Mar 14 19:48:52 wazuh-server sshd[5329]: Failed password for wazuh-user from 192.168.56.107 port 57488 ssh2
Mar 14 19:48:50 wazuh-server sshd[5327]: Failed password for wazuh-user from 192.168.56.107 port 57476 ssh2
Mar 14 19:48:50 wazuh-server sshd[5327]: Failed password for wazuh-user from 192.168.56.107 port 57476 ssh2
Mar 14 19:48:49 wazuh-server sshd[5325]: Failed password for wazuh-user from 192.168.56.107 port 57466 ssh2
t rule.description CRITICAL: SSH Brute Force Attempt Detected (Multiple failures from same IP)
rule.firedtimes 2
rule.frequency 6
t rule.groups local, syslog, sshd
t rule.id 100011
rule.level 12

Rule 3: Unauthorized USB Insertion

To detect physical data exfiltration attempts (MITRE T1052 — Exfiltration Over Physical Medium), Rule 100012 was created to elevate standard USB device mount log entries into a Level 8 Warning alert. This is particularly important in environments where removable media poses a significant insider-threat risk.



Verification: Due to agent-side operating system logging constraints, the wazuh-logtest backend engine was used to validate the rule logic. Injecting a raw kernel USB log event successfully triggered Phase 3 of the testing engine, confirming the rule was operational and correctly matching the intended log pattern.

```
<!-- Custom Rule 3: Unauthorized USB plugged in -->
<rule id="100012" level="8">
 <if_sid>81100</if_sid>
 <description>WARNING: Unauthorized USB Storage Device Plugged In</description>
 <mitre>
  <id>T1052</id>
 </mitre>
</rule>
```

Days 12 & 13 — Log Analysis Fundamentals

Log analysis forms the backbone of incident response. This phase focused on querying the Wazuh SIEM to correlate events across multiple monitored agents and answer the fundamental investigative questions: who initiated an action, what was done, and when did it happen? Three separate log sources and event types were examined and analyzed in detail.

Log Analysis Breakdown

Source OS: Linux

Event / Rule ID: Wazuh Rule 5760

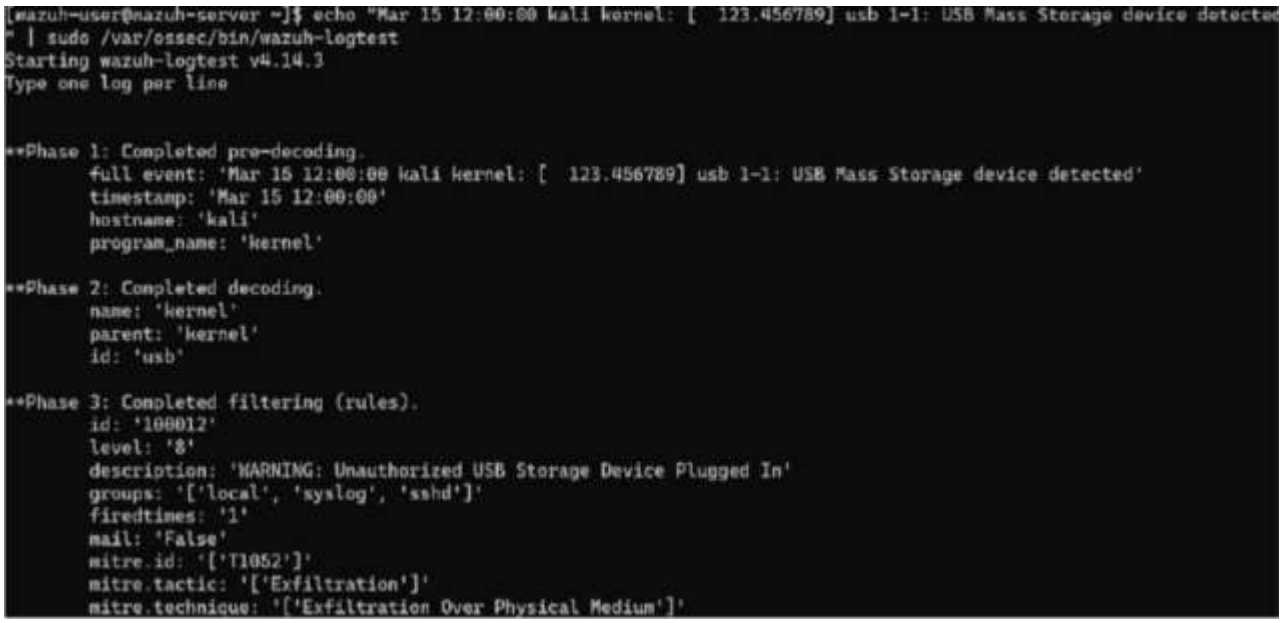
Trigger Activity

Multiple sshd: authentication failed events were observed for the user wazuh-user originating from the source IP 192.168.56.107.

Analyst Interpretation

Deliberate Attack: The frequency and volume of these failures rule out a simple typo. The pattern is consistent with a targeted SSH Brute Force attack, likely automated using a credential list or tool.

Screenshot Evidence



Source OS: Windows

Event / Rule ID: Windows Event ID 4672 (Special Privileges Assigned to New Logon)

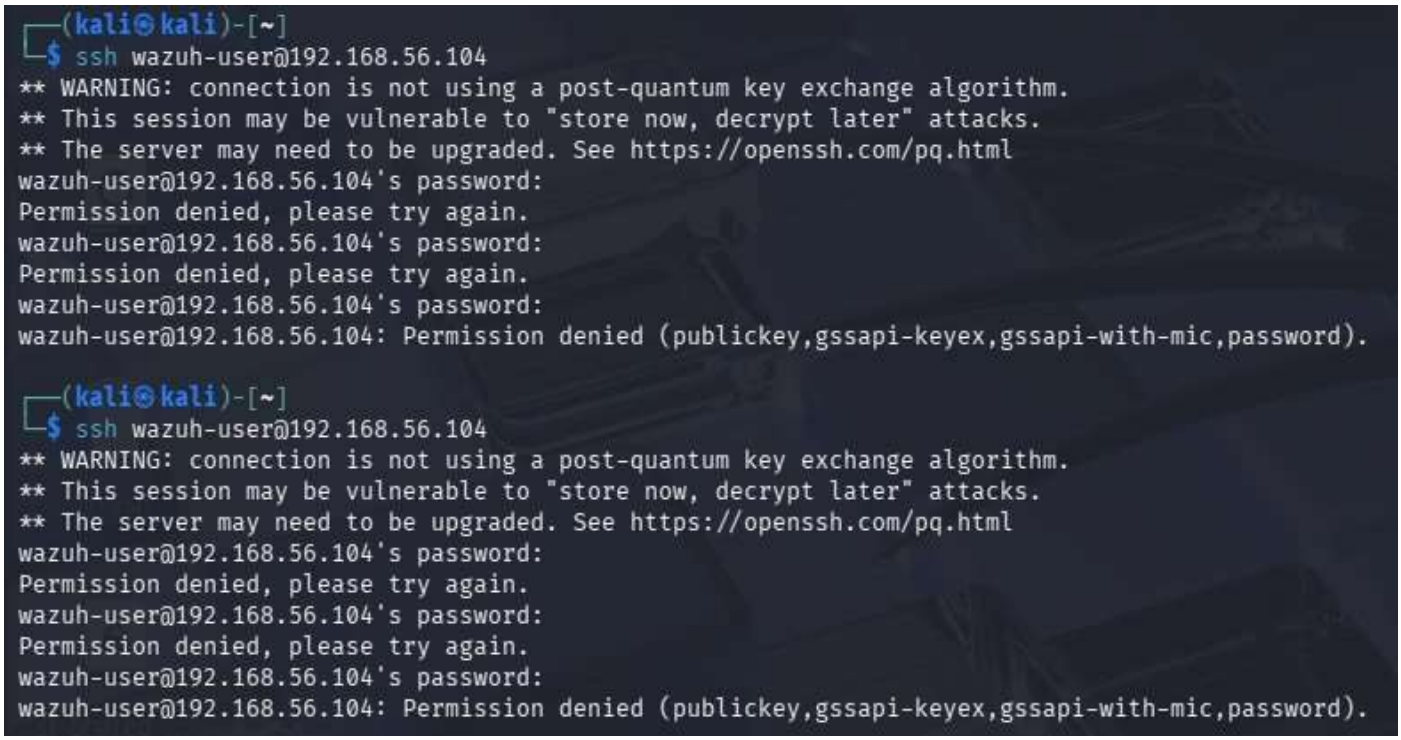
Trigger Activity

An elevated Command Prompt session assigned special privileges to a new logon session on the Windows 10 machine.

Analyst Interpretation

Suspicious if Unplanned: This event is routine if performed by a known system administrator. However, when correlated with a preceding brute-force alert from the same machine, this event becomes highly suspicious and indicative of a successful compromise and privilege escalation.

Screenshot Evidence



Source OS: Windows

Event / Rule ID: Windows Event ID 4720 (User Account Created)

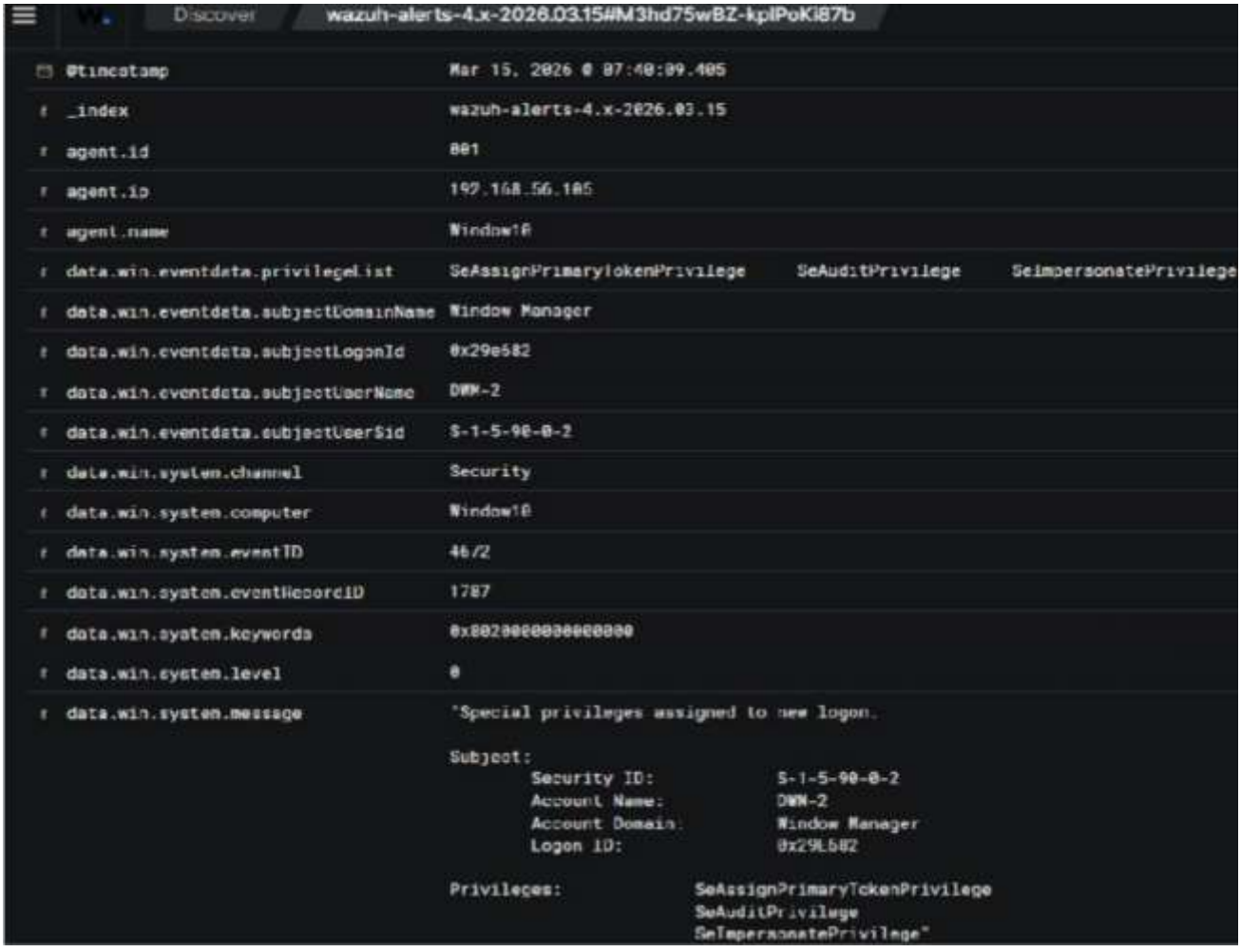
Trigger Activity

The command net user SneakyHacker was executed in an elevated prompt, creating a new local user account.

Analyst Interpretation

Critical: This is an active establishment of persistence. The logs clearly identify vboxuser as the account that created the SneakyHacker user — providing direct attribution for the backdoor creation activity.

Screenshot Evidence



Source OS: Windows

Event / Rule ID: Windows Event ID 4625 (Failed Logon Attempt)

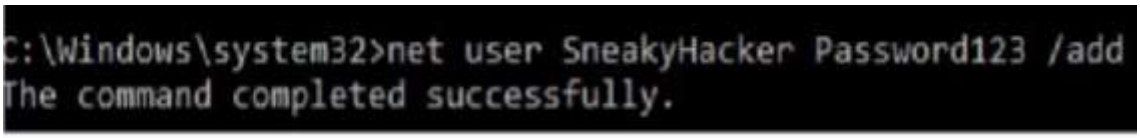
Trigger Activity

User vboxuser failed a logon attempt with SubStatus code 0xc000006a.

Analyst Interpretation

Routine (Isolated): The sub-status code 0xc000006a specifically indicates a bad password was entered. As an isolated event, this is consistent with a simple user typo. However, if this event appeared repeatedly or in conjunction with privilege escalation events, it would be re-classified as malicious.

Screenshot Evidence



Timeline of Suspicious Activity

The following table presents a chronological reconstruction of flagged security incidents identified across the monitored environment during this reporting period.

Timestamp	Event Description	Agent	Analyst Note
Mar 14, 18:34	CRITICAL: New user account created (Level 10)	Kali Linux	Flagged as potential backdoor creation (User: darthvader)
Mar 14, 19:48	CRITICAL: SSH Brute Force Attempt (Level 12)	Wazuh Server	Active credential stuffing attack from 192.168.56.107
Mar 15, 07:34	User account enabled or created (Level 8)	Windows 10	Windows Event ID 4720 triggered; persistence suspected
Mar 15, 07:40	Special privileges assigned to new logon (Level 3)	Windows 10	Windows Event ID 4672 triggered; potential privilege escalation

Day 14 — Wazuh Active Response

Active Response elevates the Wazuh SIEM from a passive monitoring tool to a fully automated defensive mechanism. Rather than simply alerting an analyst to an ongoing attack, Active Response enables the system to autonomously execute countermeasures — such as blocking attacker IPs at the host firewall — in real time, significantly reducing the window of exposure.

For this task, Wazuh was configured to automatically drop all network traffic originating from any IP address that triggered the custom SSH Brute Force rule (Rule 100011), using the built-in firewall-drop script.

Configuration Workflow

- Tuned Custom Rule 100011 to trigger on exactly 5 failed attempts (frequency="5") to ensure the response fires promptly.

```
<!-- Custom Rule 2: SSH Brute Force -->
<rule id="100011" level="12" frequency="5" timeframe="120">
 <if_matched_sid>5760</if_matched_sid>
 <same_source_ip />
 <description>CRITICAL: SSH Brute Force Attempt Detected (Multiple failures from same IP)</description>
 <mitre>
  <id>T1110.001</id>
 </mitre>
</rule>
```

- Created a backup of the master Wazuh configuration file before making any changes: sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak

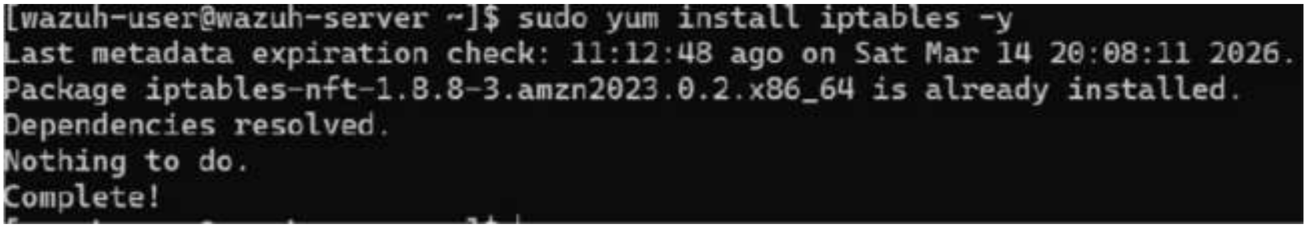
```
[wazuh-user@wazuh-server ~]$ sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak
```

- Edited the <active-response> block in ossec.conf to utilize the firewall-drop command. The timeout was set to 300 seconds (5 minutes) to allow safe testing while demonstrating the full block-and-restore cycle.

```
[wazuh-user@wazuh-server ~]$ sudo nano /var/ossec/etc/ossec.conf
```

```
<!-- Active Responses -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100011</rules_id>
  <timeout>300</timeout>
</active-response>
```

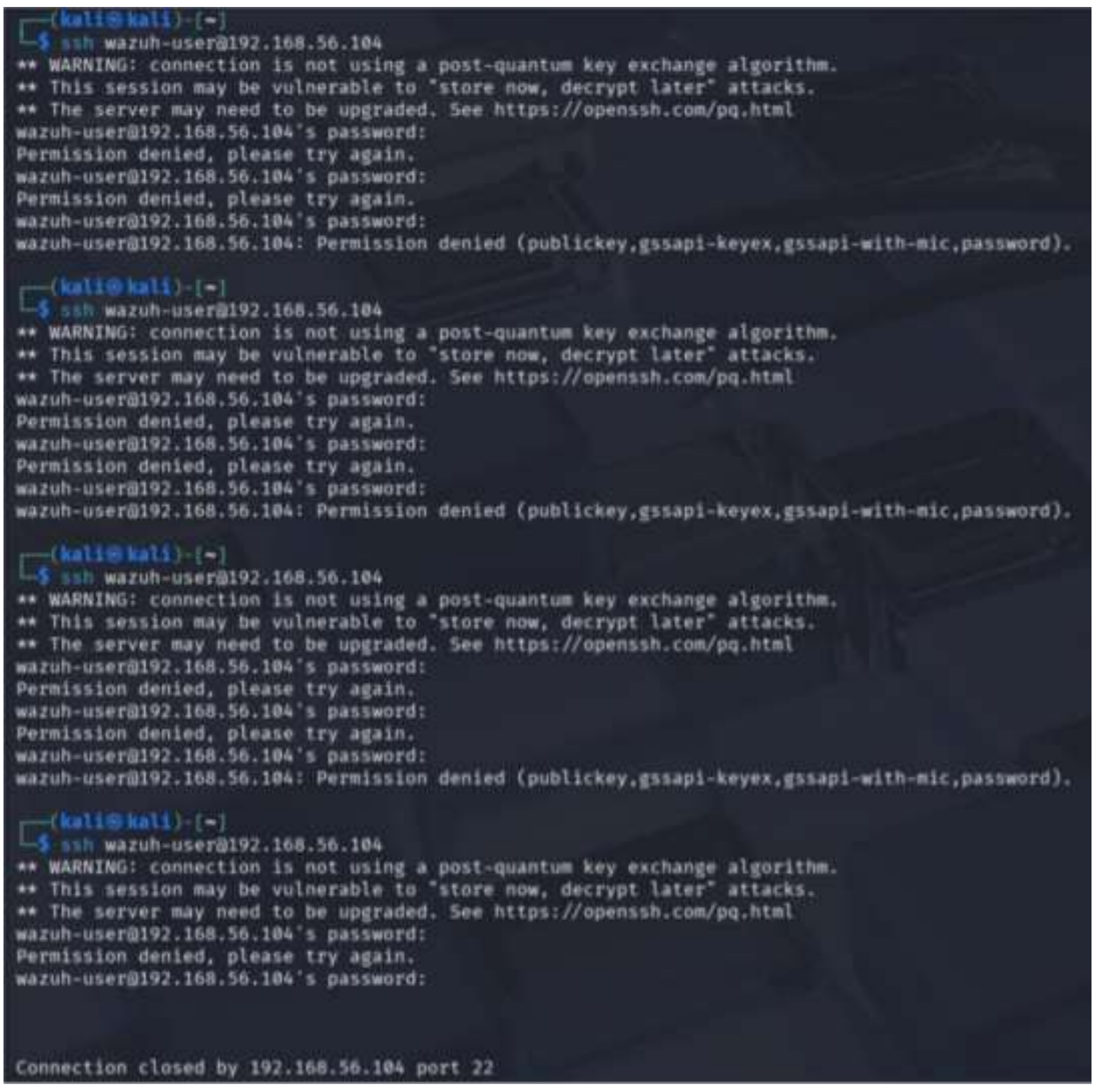
- Verified that the iptables firewall utility was pre-installed on the Amazon Linux server to confirm the firewall-drop script could execute successfully.



```
[wazuh-user@wazuh-server ~]$ sudo yum install iptables -y
Last metadata expiration check: 11:12:48 ago on Sat Mar 14 20:08:11 2026.
Package iptables-nft-1.8.8-3.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
```

Attack & Verification

An SSH brute-force attack was initiated from the Kali Linux machine targeting the Wazuh server. On the 6th failed attempt, the terminal session froze completely and the connection was refused — confirming that the Active Response had executed and the attacker's IP had been blocked at the firewall level.



```
┌──(kali㉿kali)-[~]
└─$ ssh wazuh-user@192.168.56.104
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
wazuh-user@192.168.56.104: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).

┌──(kali㉿kali)-[~]
└─$ ssh wazuh-user@192.168.56.104
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
wazuh-user@192.168.56.104: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).

┌──(kali㉿kali)-[~]
└─$ ssh wazuh-user@192.168.56.104
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:
wazuh-user@192.168.56.104: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).

┌──(kali㉿kali)-[~]
└─$ ssh wazuh-user@192.168.56.104
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
wazuh-user@192.168.56.104's password:
Permission denied, please try again.
wazuh-user@192.168.56.104's password:


Connection closed by 192.168.56.104 port 22
```

Running sudo iptables -L -n on the server confirmed that the Wazuh agent had successfully written a DROP rule to the host firewall, actively blocking all inbound traffic from the attacker's IP address (192.168.56.107).

```
[wazuh-user@wazuh-server ~]$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.56.107       0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.56.107       0.0.0.0/0

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

The Wazuh dashboard provided a complete view of the full kill chain: the individual login failures were correlated and grouped into a Brute Force alert, which was instantly followed by Rule 651 confirming Active Response execution — all visible in the unified event stream.

Risks & Safeguards of Active Response

While Active Response is a powerful capability, deploying it in a production environment carries significant operational risks that must be carefully managed.

Primary Risk — Self-Denial of Service (DoS): If a legitimate administrator forgets their password and makes multiple failed login attempts, an overly aggressive Active Response configuration could lock them out of a production server entirely.

Secondary Risk — IP Spoofing Attack: A sophisticated attacker can forge their source IP to match a mission-critical internal asset (e.g., a Domain Controller or primary DNS server), intentionally triggering the brute-force rule and causing Wazuh to automatically ban the company's own infrastructure — effectively taking the network offline.

Recommended Safeguards:

- Configure a robust <white_list> in ossec.conf containing all mission-critical IP addresses — Domain Controllers, vulnerability scanners, and administrator workstations — to prevent them from ever being auto-banned.
- Avoid permanent bans in production. Use a timeout threshold of 5–15 minutes so that if an accidental block occurs, the system self-heals and restores access automatically without requiring emergency intervention.
- Correlate Active Response events in the dashboard to quickly identify and investigate any unexpected blocks.

— End of Week 2 Report —